

WASM / REDIS / REACT

WasmRedis

Une base Redis lite, entierement dans le navigateur

Go en RAM. OPFS sur disque. React reste fluide.

React

SDK TS

Worker

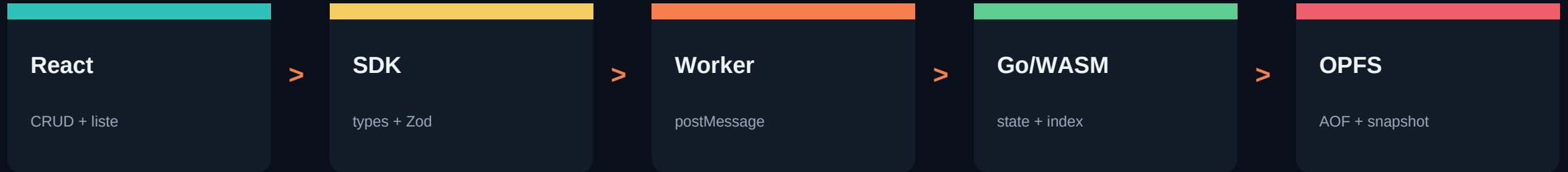
Go/WASM

OPFS

Une commande, cinq étapes

02

Le fil rouge de toute la presentation



Cycle de persistance



React ne bloque jamais : le moteur et le disque vivent dans le worker.

La RAM donne la vitesse ; AOF + snapshot donnent la persistance.

Le parser protege l'entree

03

Du texte vers une commande Go valide

`internal/redis/parser.go:44-51`

```
44     switch CommandType(commandName) {
45
46     // GET accepte soit une cle, soit la forme GET WHERE.
47     case CommandGet:
48         if len(parts) > 1 && strings.EqualFold(parts[1], "WHERE") {
49             return parseWhereCommand(parts)
50         }
51         return parseKeyCommand(CommandGet, parts)
```

GET WHERE est distingue d'un GET par cle.

Toute commande inconnue est rejetee avant le moteur.

Le buffer appartient au moteur Go

04

Les écritures attendent en RAM avant le flush

[internal/redis/engine.go:230-239](#)

```
230 func (engine *Engine) DrainBuffer() []Operation {
231     engine.bufferMu.Lock()
232     defer engine.bufferMu.Unlock()
233
234     writes := make([]Operation, len(engine.buffer))
235     copy(writes, engine.buffer)
236     engine.buffer = engine.buffer[:0]
237     return writes
238 }
239
```

Le mutex interdit deux vidages concurrents.

DrainBuffer copie les opérations puis remet la file à zéro.

Le worker serialise l'accès OPFS

web/src/worker/wasmRedis.worker.ts:386-400

```
386 async function writePendingAof(): Promise<void> {
387   const writes = retryAof.concat(drainGoBuffer());
388   if (writes.length === 0) {
389     return;
390   }
391
392   retryAof = [];
393   const lines = writes.map((operation) => JSON.stringify(operation)).join("\n") + "\n";
394   try {
395     await appendOpfsText(aofFileName, lines);
396   } catch (error) {
397     retryAof = writes.concat(retryAof);
398     throw error;
399   }
400 }
```

L'AOF est ajoutée en fin de fichier, jamais réécrite à chaque SET.

En cas d'échec OPFS, les opérations retournent dans `retryAof`.

Les plages passent par le B-Tree

06

Le parcours s'arrete des que la limite est depassee

[internal/redis/btree.go:95-103](#)

```
95     if operator == OperatorLessThan || operator == OperatorLessOrEqual {
96         tree.walkAscending(tree.root, func(item BTreeItem) bool {
97             if !matchesRange(item.Value, operator, target) {
98                 return false
99             }
100             items = append(items, item)
101             return true
102         })
103     return items
```

equals utilise un index inverse ; contains assume un scan.

Une plage selective reste a environ 0,3 us jusqu'a 10 000 entrees.

Le TTL supprime vraiment la cle

07

State, index et AOF restent cohérents

[internal/redis/engine.go:451-459](#)

```
451 func (engine *Engine) removeIfExpiredLocked(key string, stored StoredValue) bool {
452     if !engine.isExpired(stored) {
453         return false
454     }
455
456     delete(engine.state, key)
457     engine.removeFromEqualsIndexLocked(key, stored.Value)
458     engine.removeFromNumberIndexLocked(stored.Value)
459     return true
```

Expiration lazy au GET, plus balayage actif periodique.

Le restore garde ExpiresAt et ne remplit pas de nouveau le buffer.

Le SDK rend les requetes typees

08

Les erreurs simples sont trouvees avant l'execution

web/src/sdk/wasmRedis.ts:35-42

```
35 export type WhereMethod<Schema extends Record<string, unknown>> = {
36   <Key extends RedisKey<Schema>>(
37     field: Key,
38     operator: FilterOperatorFor<Schema[Key]>,
39     value: Schema[Key],
40   ): WhereQuery<Schema>;
41   (field: FilterField, operator: FilterOperator, value: string | number): WhereQuery<Schema>;
42 };
```

age: number autorise les plages ; name: string autorise contains.

Zod revalide aussi les donnees au runtime avant postMessage.

100 000 entrees, 29 lignes DOM

09

Virtual scroll maison + abonnement cible

[web/src/virtual/getVirtualRange.ts:22-27](#) | [web/src/store/entryStore.ts:60-64](#)

```
22   const firstVisible = Math.min(itemCount - 1, Math.floor(scrollTop / rowHeight));
23   const startIndex = Math.max(0, firstVisible - overscan);
24   const visibleCount = Math.ceil(viewportHeight / rowHeight) + overscan * 2;
25   const endIndex = Math.min(itemCount, startIndex + visibleCount);
26
27   return { startIndex, endIndex, totalHeight: itemCount * rowHeight };

60 export function useEntry(key: string): StoreEntry | undefined {
61   const subscribe = useCallback((listener: Listener) => subscribeEntry(key, listener), [key]);
62   const getSnapshot = useCallback(() => getEntry(key), [key]);
63   return useSyncExternalStore(subscribe, getSnapshot, getSnapshot);
64 }
```

Le spacer simule la hauteur totale ; seule la tranche visible est montee.

Modifier une ligne : compteur 1 -> 2 ; autre ligne : 1 -> 1.

Ce que la demo prouve

10

Mesures reelles, parcours reproductible

15,5 x

gain batch / 30 SET

~0,3 us

range selective / 10k

60 FPS

scroll mesure

29

lignes DOM / 100k

Demo en 60 secondes

- 1 SET avec TTL, puis Flush
- 2 Reload : restore OPFS
- 3 Seed 100, filtre ≥ 50
- 4 Editer : 1 -> 2
- 5 Lancer Benchmark

Validation finale

Go race tests	OK
Tests TypeScript	OK
Build WASM + React	OK